Coalgebra, Categories, Automota, and MDPs

lorisj

May 2025

1 Introduction

The main goal of these notes is to explore new ideas in the intersection of:

- Category theory/coalgebra
- MDPs (Markov Decision Processes)
- LDBA (Limit Deterministic Buchi Automota),
- Bisimulation and bisimulation metrics

and also serve as a reference for some results/definitions in these areas.

2 Notation

Definition: Notation Used

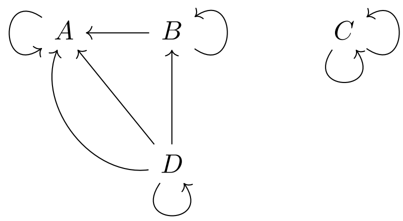
These are default notations that are used unless otherwise noted:

- TFDC $\stackrel{\text{def}}{=}$ The following diagram(s) commute(s).
- s.t. $\stackrel{\text{def}}{=}$ such that
- $A^B \stackrel{\text{def}}{=} \{f : B \to A\}$ for sets A, B.
- $2^A = \mathcal{P}[A]$ (powerset of A) for set A, note that this means the powerset can be interpreted as the set of functions from $A \to 2$.
- $\omega \stackrel{\text{def}}{=} \{\text{natural numbers including } 0\} = \{0, 1, 2, \ldots\}$.
- $\forall_v^V, \exists_v^V \stackrel{\text{def}}{=} \forall_{v\in V}, \exists_{v\in V}$, where the superscript refers to the set being quantified over (domain) and the subscript denotes the variable bound by the quantifier.
- $f[X], f^{-1}[X] \stackrel{\text{def}}{=}$ image, preimage for a set X under function f (in contrast $f(x), f^{-1}(x)$ denote element application)
- $(\cdot)$ denotes a unique free variable (i.e. $(\cdot) + 2$ is the function that takes in a variable and adds 2)
- ΔX DEF `distribution-functor`

3 Category Theory

3.1 Categories

Intuitively, a category can be thought of as a more structured graph, where you can compose edges. Specifically, a category is a directed multigraph, and also there is a composition operation, as well as an identity. The nodes are called Objects, and the edges are called arrows (or morphisms). Essentially a category will look something like this:



The important notes are that:

1. You can have multiple arrows between two nodes that are not equal
2. If I have two arrows: $f : A \to B, g : B \to C$, then there must exist some arrow $g \circ f : A \to C$.
3. There exists at least one self loop for each object, called the identity, $\text{id}_c : C \to C$ such that its composition with any other object does not change it.

More formally:

Definition: Category `category`

A category C consists of:

- $\textbf{Obj}(C)$: A class of objects
- $\textbf{Hom}(C)$: A class of morphisms (sometimes called arrows)
- $\circ$: A class function : $\textbf{Hom}(C) \times \textbf{Hom}(C) \to \textbf{Hom}(C)$

Such that:

1. Each morphism has a *domain* and *codomain*:

   For every $f \in \textbf{Hom}(C)$ there must be two objects, a *domain* and *codomain*, both in $\textbf{Obj}(C)$.

   As notation, $f : X \to Y$ denotes a morphism with domain X, codomain Y, and $\textbf{Hom}(X, Y)$ for objects X, Y denotes the class of all morphisms with domain X, codomain Y.

2. Composition is *type-respecting*:

   If $f : X \to Y$ and $g : Y \to Z$, then the composite $g \circ f$ must exist in $\textbf{Hom}C$, and must be of type: $g \circ f : X \to Z$.

3. Composition is *associative*:

   For morphisms $f : W \to X$, $g : X \to Y$, and $h : Y \to Z$:

$$(h \circ g) \circ f = h \circ (g \circ f) \tag{1}$$

4. *Identity morphisms* exist:

   Every $X \in \textbf{Obj}(C)$ has a *identity* (or *unit*) morphism $\text{id}_X : X \to X$ such that for every morphism $f : X \to Y$ and $g : Z \to X$:

$$\text{id}_Y \circ f = f \tag{2}$$

$$f \circ \text{id}_X = f \tag{3}$$

This definition is very abstract, so the following examples should make the idea more concrete.

Example: Common categories

Here are some common categories: (See DEF category)

Set, the category of sets : catset

- **Obj**: All sets.
- **Hom**: All functions between sets.
- $\circ$: Normal function composition.
- id_X: The identity function on a set X.

Rel, the category of relations : catrel

- **Obj**: All sets.
- **Hom**: A morphism $R : X \to Y$ is a binary relation $R \subseteq X \times Y$.
- $\circ$: For $R : X \to Y$ and $S : Y \to Z$, their composite $S \circ R : X \to Z$ is given by:

$$(x, z) \in (S \circ R) \quad \text{iff} \quad \exists y \in Y \text{ such that } (x, y) \in R \text{ and } (y, z) \in S.$$

- id_X: For each set X, $\mathrm{id}_X = \{(x, x) : x \in X\}$, i.e. the diagonal relation.

Typ, the category of types : cattypes

- **Obj** : Types in a given type theory or programming language.
  (think `string`, `int`, `double`, `string[]`)
- **Hom** : Pure functions (i.e. programs with no side effects) from the input type to the output type (think length : string $\to$ number). Note only pure functions are allowed, so something that has side effects would not work.
- $\circ$: Composition of functions.
- id_X : Identity function on type X.

Grp, the category of groups : catgroup

- **Obj**: All groups.
- **Hom**: Group homomorphisms. Formally, a morphism between groups $f : (G, \cdot^G) \to (H, \cdot^H)$ respects the group operation: $f(g_1 \cdot^G g_2) = f(g_1) \cdot^H f(g_2)$.
- $\circ$: Composition is the usual function composition of homomorphisms.
- $\mathrm{id}_{(G,\cdot)}$: The identity on a group $(G, \cdot)$ is the identity homomorphism id_G which sends each $g \in G$ to itself.

The following example shows that morphisms on a category need not reduce to relations on the class of objects:

Category of Dimensions of Linear maps :

- **Obj**: The natural numbers ω.
- **Hom**: A morphism $R : n \to m$ is a $n \times m$ matrix $\in \mathbb{R}^{n \times m}$.
- $\circ$: matrix multiplication.
- id_n: For $n \in \omega$, id_n is simply the $n \times n$ identity matrix.

3.2 Basic Categorical Structures

Here are some basic categorical structures that we will use, and these should give some intuition as to what a categorical definition looks like.

3.2.1 Span and Cospan

Maybe the easiest objects to define.

Definition: Span and cospan `span-cospan`

For a category C:

Span : `span`

A span consists of a diagram of the form:

$$A \xleftarrow{f} B \xrightarrow{g} C$$

Cospan : `cospan`

A cospan consists of a diagram of the form:

$$A \xrightarrow{f} B \xleftarrow{g} C$$

3.2.2 Monoid

If we look at a category, sometimes we can see an isolated object, i.e. something that looks like this:

$$\begin{tikzcd} A \arrow[loop above, "f"] \arrow[loop right, "g"] \arrow[loop left, "k"] \end{tikzcd}$$

where there are no arrows going from A outside of A. This is called a monoid. More formally:

Definition: Monoid `monoid`

A monoid in a category C is an object M such that every morphism f with domain M must also have codomain M.

Recall the traditional definition of a monoid consists of:

Set theoretic monoid : A monoid $(M, +, e)$ is a set M, function $+ : M \times M \to M$, element $e \in M$ s.t.:

1. $+$ is associative
2. For all $m \in M$: $e + m = m + e = m$

Category Theory Monoid is Set Monoid : Let $* \in \mathbf{Obj}(C)$ be a monoid. Then $(\mathbf{Hom}(*, *), \circ)$ is a set theoretic monoid. (recall $\circ$ is the composition operator of C)

Proof. Associativity and the existence of identity follow immediatly from the category theory axioms. □

3.2.3 Isomorphism and Isomorphic objects

An isomorphism generalizes the idea of a bijection. Namely:

Definition: Isomorphism `isomorphism`

For a category C, an isomorphism is any morphism $f : X \to Y$ such that there exists another morphism $f^{-1} : Y \to X$ such that $f^{-1} \circ f = \mathrm{id}_X$, and $f \circ f^{-1} = \mathrm{id}_Y$. As a diagram:

$$\begin{tikzcd} X \arrow[r, bend left, "f"] & Y \arrow[l, bend left, "f^{-1}"] \end{tikzcd}$$

We can transfer this definition from morphisms to objects:

Definition: Isomorphic objects `isomorphic`

In a category C, objects $X, Y \in C$ are isomorphic (denoted $X \cong Y$) if there exists some isomorphism $f : X \to Y$.

Isomorphisms/Isomorphic objects :

- In **Set**, isomorphisms are exactly bijective functions, and two sets are isomorphic iff they have the same cardinality.
- In **Grp**, isomorphims are group homomorphisms that are invertible, i.e. group isomorphisms.

Many categorical definitions are only unique up to isomorphism, meaning that any two objects that satisfy the condition must be isomorphic. For example see RES `terminal-unique`.

3.2.4 Initial and Terminal/Final Objects

Definition: Initial Object `initial-obj`

An object $* \in \mathbf{Obj}(C)$ for some category C is called initial if for any other object $X \in \mathbf{Obj}(C)$, there is a unique morphism $! : * \to X$.

Definition: Terminal/Final Object `terminal-obj`

An object $* \in \mathbf{Obj}(C)$ for some category C is called terminal (aka final) if for any other object $X \in \mathbf{Obj}(C)$, there is a unique morphism $! : X \to *$.

Example: Initial/Terminal Objects

Initial for **Set** : Note that if we look at **Set**, the set with a single element (usually denoted $1 = \{0\} = \{\emptyset\}$) is terminal. (For any other set, there is a unique map that just sends everything to 0)

Terminal for **Set** : By convention, we assume that there is a unique function from any set to the emptyset. This means that the empty set (aka 0) is the terminal object in **Set**.

Terminal for **Grp** : Note that the trivial group $(\{0\}, +, 0)$ is terminal in **Grp**. Namely for any other group $(G, +, 0)$ there is only one homomorphism to the trivial group, namely the one that sends everything to the identity.

Result: Terminal Objects are Unique up to Isomorphism `terminal-unique`

Let $*_0, *_1$ both be terminal objects of a category C. Then $*_0 \cong *_1$.

Proof. Because $*_0$ is terminal, there must be some (unique) $!_0 : *_1 \to *_0$. Likewise because $*_1$ is terminal, there must be some (unique) $!_1 : *_0 \to *_1$. So notice that $!_0 \circ !_1 : *_0 \to *_0$, so by finality of $*_0$ the identity map is the unique morphism $: *_0 \to *_0$, so $!_0 \circ !_1 = \mathrm{id}_{*_0}$. The same argument holds for $!_1 \circ !_0 = \mathrm{id}_{*_1}$. Thus by DEF `isomorphism` $!_0$ is an isomorphism, and so $*_0 \cong *_1$. □

Note that the same is true for initial objects. This means that we can say **the** [initial/terminal] object instead of **an/a** [initial/terminal] object.

3.2.5 Product and Coproduct

Definition: Product `product`

Let $A, B \in \mathbf{Obj}(C)$ for some category C. A product consists of:

- An object $A \times B \in \mathbf{Obj}(C)$
- A morphism $\pi_A : A \times B \to A$
- A morphism $\pi_B : A \times B \to B$

s.t. for any other span between A and B: $A \xleftarrow{f} Q \xrightarrow{g} B$ (DEF `span`) we have that there exists a unique map $r_Q : Q \to A \times B$ s.t. TFDC:

$$\begin{tikzcd} & A \times B \arrow[dl, bend right, "\pi_A"'] \arrow[dr, bend left, "\pi_B"] & \\ A & Q \arrow[l, "f"] \arrow[r, "g"'] \arrow[u, dashed, "\exists! r_Q"] & B \end{tikzcd}$$

Intuitively, this means that if I know a product exists, and I have some span $A \xleftarrow{f} Q \xrightarrow{g} B$, we have that $f = \pi_A \circ r_Q$, and $g = \pi_B \circ r_Q$. This means that we can factor/reduce/simplify f, g by going through the product.

IMPORTANT NOTE: Not all categories have a product for every two objects, however many of the ones we will see do.

Products of some categories : The product for objects $A, B \in \mathbf{Obj}(C)$ in the categories C below is given by:

Set : The product of sets A, B consists of the cartesian product $\{(a,b) | a \in A, b \in B\}$, where π_A, π_B are the projection functions from $A \times B$ to A and B respectfully. Note that given a span $A \xleftarrow{f} Q \xrightarrow{g} B$, the unique r_Q $\left(\text{see } \texttt{DEF product}\right)$ is given by $r_Q = (q : Q) \mapsto (f(q), g(q))$.

Grp : The product of groups $(G_0, +_0, e_0)$, $(G_0, +_1, e_1)$ is a group $(G_0 \times G_1, (+_0, +_1), (e_0, e_1))$, where $G_0 \times G_1$ is the cartesian product on sets, and $(+_0, +_1)$ acts elementwise.

Definition: Coproduct `coproduct`

Let $A, B \in \mathbf{Obj}(C)$ for some category C. A coproduct consists of:

- An object $A + B \in \mathbf{Obj}(C)$
- A morphism $i_A : A \to A + B$
- A morphism $i_B : B \to A + B$

s.t. for any other cospan between A and B: $A \xrightarrow{f} Q \xleftarrow{g} B$ $\left(\texttt{DEF cospan}\right)$ we have that there exists a unique map $r_Q : A + B \to Q$ s.t. TFDC:

$$\begin{tikzcd}
& A+B \arrow[d, dashed, "\exists! r_Q"'] & \\
A \arrow[ur, bend left, "i_A"] \arrow[r, "f"'] & Q & B \arrow[ul, bend right, "i_B"'] \arrow[l, "g"]
\end{tikzcd}$$

Coproduct :

Set : The coproduct of sets A, B consists of the disjoint union $A \sqcup B$, where the $i_A : A \to A \sqcup B$ is the inclusion map of A, and $i_B : B \to A \sqcup B$ is the inclusion map of B. Note that for any cospan $A \xrightarrow{f} Q \xleftarrow{g} B$, the unique r_Q $\left(\text{see } \texttt{DEF coproduct}\right)$ is given by $r_Q = (q : Q) \mapsto \begin{cases} f(q) & q \in A \\ g(q) & q \in B \end{cases}$

3.3 Duality

You may have noticed many of these definitions look the same, but are sort of opposite to each other. For instance the definition of a coproduct is the same as a product, but where all of the arrows are flipped. Likewise a cospan is a span but with all arrows are flipped. This notion is called duality, many times in category theory there is some definition that is of interest, and it turns out that usually, the dual definition (consisting of reversing all of the arrows) usually describes something interesting as well. When we have a name for the base thing, and we want a name for the dual, we just add the word co- in front. (coproduct, cospan, colimit, coalgebra, comonad, . . .)

3.4 Functors

A functor is sort of like a homomorphism between categories. A functor should take an object to another object, and an arrow should take an arrow to another arrow. It should also preserve composition, as well as identity morphisms. More formally:

Definition: Functor `functor`

Let C, D be two categories. A (covariant) *functor* $F : C \to D$ consists of:

- A class function on objects:
  For each object $X \in \mathbf{Obj}(C)$, there must exist an object $FX \in \mathbf{Obj}(D)$
- A class function on morphisms:
  For each morphism $f : X \to Y$ in $\mathbf{Hom}(C)$, there must exist a morphism $Ff : FX \to FY$.

These assignments must satisfy the following conditions:

- *Preservation of* id:
$$F(\mathrm{id}_X^C) = \mathrm{id}_{FX}^D \quad \text{for all } X \in \mathbf{Obj}(C) \tag{4}$$
- *Preservation of* $\circ$:
$$F(g \circ^C f) = F(g) \circ^D F(f) \tag{5}$$

Intuitivly, a functor basically just adds F in front of everything. In particualar, if you have any diagram that commutes:

$$\begin{tikzcd} & Y \arrow[dr, "h"] & \\ \cdots \arrow[r] & X \arrow[u, "f"] \arrow[r, "g"] \arrow[r, bend right, "k"'] & Z \end{tikzcd}$$

then after lifting the diagram (applying F everywhere) it should still commute:

$$\begin{tikzcd} & FY \arrow[dr, "Fh"] & \\ \cdots \arrow[r] & FX \arrow[u, "Ff"] \arrow[r, "Fg"] \arrow[r, bend right, "Fk"'] & FZ \end{tikzcd}$$

Definition: Endofunctor `endofunctor`

An endofunctor is simply a functor that has the same codomain, namely an endofunctor on a category C is a functor $F : C \to C$.

Example: Functor Examples

On **Set** : , these are functors, which take a set X and a morphism $f : X \to Y$ to:

- $FX = X$, and $F(f) = f$. This is called the identity functor.
- $FX = X^2$, and $F(f) = (f, f) : X^2 \to Y^2$.
- $FX = X + A$ for some fixed set A, where a morphism $f : X \to Y$ maps to:

$$F(f) : X + A \to Y + A \tag{6}$$

$$F(f) : (u : X + A) \mapsto \begin{cases} f(u) & u \in X \\ u & u \in A \end{cases} \tag{7}$$

- $FX = 2^X$, and $F(f) = f[\cdot]$. (also called the covariant powerset functor)
- The distribution functor:

Definition: Distribution Functor `distribution-functor`

The distribution functor on **Set**:

- On objects: maps a set X to the set

$$\Delta X = \{P : X \to [0, 1]_{\mathbb{R}} \mid \sum_{x \in X} P(x) = 1, P \text{ has finite support}\} \tag{8}$$

- On morphisms: maps a function $f : X \to Y$ to the function $\Delta f : \Delta X \to \Delta Y$, taking $P \mapsto (y : Y) \mapsto \sum_{x \in f^{-1}[\{y\}]} P(x)$.

Functors for programmers : If you come from a programming background, the easiest way of thinking about a functor is of a generic type. Recall **Typ** from DEF `cattypes`. A functor on **Typ** consists of a generic type/way to lift types (a function : $\texttt{Type} \to \texttt{Type}$), as well as a way to lift functions. The typical example is the list functor:

$$: \texttt{X} \mapsto \texttt{X[]}$$

On morphisms, given a function/program $f : \texttt{X} \to \texttt{Y}$, you get the function $\texttt{map}(f) : \texttt{X[]} \to \texttt{Y[]}$, which just applys f elementwise. For example take the function: $\texttt{len} : \texttt{string} \to \texttt{Nat}$. This function takes in a string, and returns it's length as a natural number. The lifted function $\texttt{map(len)}$ will take in a string array, and output each strings length:

$$\texttt{map(len)} : \texttt{string[]} \to \texttt{Nat[]} \tag{9}$$

$$: \texttt{["abc", "a", ""]} \mapsto \texttt{[3, 1, 0]} \tag{10}$$

If you are familiar with generic types, then a functor is exactly a generic type (usually written as `T<X>`) as well as a way to lift maps to the generic type.

3.5 Coalgebra

As it turns out, many systems can be modeled in terms of category theory, using coalgebras (which we will define shortly). In fact the standard notion of bisimulation and trace equivalence can be modeled canonically using fairly simple definitions, and because of how general the language of category theory is, it allows us to come up with canonical notions of bisimulation for many different systems. For now we have to define what a system is.

Definition: Coalgebra `coalgebra`

A coalgebra for an endofunctor $F : C \to C$ is a morphism

$$c_X : X \to FX \tag{11}$$

Note that there are no additional constraints. Note that if we think of F adding structure to X, then a coalgebra can model many different forms of systems:

Coalgebras on **Set** : Consider **Set**. Here are some standard functors, along with the systems that they represent:

- $F(X) = X$

  Deterministic system (each state transitions to exactly one next state).
- $F(X) = A \times X$

  Labeled deterministic system (each state emits a label in A and transitions to a unique next state).
- $F(X) = 1 + X$

  System that can either terminate (landing in the singleton 1) or transition to a next state.
- $F(X) = 1 + X + X^2$ (+ is disjoint union/coproduct of sets)

  System where each state can either terminate, transition deterministically, or offer two next states, equivalently a potentially infinite tree.
- $F(X) = X^A$

  System where each action $a \in A$ leads to a next state (an "action-indexed" deterministic system).
- $F(X) = 2^X$ (also called powerset, $\mathcal{P}A$)

  System where each state has a set of next states, or equivalently a non deterministic transition system.
- $F(X) = \Delta X$ (where Δ is the distribution functor, and so ΔX is the set of distributions on X which acts on morphisms via the pushforward measure)

  System with probablistic transitions.
- $F(X) = \mathbb{R} \times (\Delta X)^A$

  Markov Decision Process (MDP) with state valued reward. See here DEF `mdp-functor` for more.

We will also need this structure:

Definition: Coalgebra Homomorphism `coalgebra-hom`

Let $c_X : X \to FX$, $c_Y : Y \to FY$ be two coalgebras.
A coalgebra homomorphism is a function $\phi : X \to Y$ such that TFDC:

$$\begin{tikzcd} X \arrow[r, "\phi"] \arrow[d, "c_X"'] & Y \arrow[d, "c_Y"] \\ FX \arrow[r, "F\phi"'] & FY \end{tikzcd}$$

Result: Coalgebras form a category

For a given functor F, the set of F-coalgebras form a category. Namely this category has:

- Objects: F-coalgebras

- Morphisms: F-coalgebra morphisms
- Composition: Function composition
- id_X: Identity functions

Proof. The only things to check is that the composition of coalgebra homomorphisms is a coalgebra homomorphism, and that there are identities (which are just the identity functions).
Assume you have 3 coalgebras c_X, c_Y, c_Z. Let $\phi : c_X \to c_Y$, and $\psi : c_Y \to c_Z$.

$$\begin{tikzcd} X \arrow[r, "\phi"] \arrow[d, "c_X"'] & Y \arrow[r, "\psi"] \arrow[d, "c_Y"] & Z \arrow[d, "c_Z"] \\ FX \arrow[r, "F\phi"'] & FY \arrow[r, "F\psi"'] & FZ \end{tikzcd}$$

The diagram commutes because both ϕ and ψ are coalgebras.
Now because F is a functor, $F\phi \circ F\psi = F(\phi \circ \psi)$, and so you can make the homomorphism : $c_X \to c_Z$:

$$\begin{tikzcd} X \arrow[r, "\psi\circ\phi"] \arrow[d, "c_X"'] & Z \arrow[d, "c_Z"] \\ FX \arrow[r, "F(\psi\circ\phi)"'] & FZ \end{tikzcd}$$

The identity part can be shown as well by just assuming $\phi = \text{id}^X, Y = X$ or $\psi = \text{id}^Y, Z = Y$. Associativity can be shown as well because coalgebra homomorphism composition is the same as function composition. □

4 Slice and product category

Later we will use the following constructions:

Definition: Slice Category `slice-cat`

Let C be some category, and let $A \in \mathbf{Obj}(C)$. The slice category C/A is the category with:

- $\mathbf{Obj}(C/A)$: V_X, where $V_X : X \to A$ is a morphism in C.
- $\mathbf{Hom}(C/A)$: $f : V_X \to V_Y$ reduces to $f : X \to Y$ s.t. TFDC:

$$\begin{tikzcd} X \arrow[rr, "f"] \arrow[dr, "V_X"'] & & Y \arrow[dl, "V_Y"] \\ & A & \end{tikzcd}$$

Set$/2$: **Set**$/2$ is the category with:

- $\mathbf{Obj}(\mathbf{Set}/2) \overset{\text{def}}{=} \{X \mid X \in \mathbf{Obj}(\mathbf{Set})\}$
- $\mathbf{Hom}(\mathbf{Set}/2) \overset{\text{def}}{=} \{m_X : X \to 2 \mid X \in \mathbf{Obj}(\mathbf{Set})\}$
- $\text{id}_X, \circ$ just like in **Set**.

Note that objects in this category are just binary partitions of $\mathbf{Obj}(\mathbf{Set})$, and morphisms are just functions that preserve the partition.
In particular if you have an object $m_X : X \to 2$, this induces a partition $X = m_X^{-1}[\{0\}] + m_X^{-1}[\{1\}]$.

Furthermore any morphism $f : m_X \to m_Y$ must preserve the partition, i.e. $m_Y(f(x)) = m_X(x)$ for all $x \in X$.

Definition: Product Category

product-cat

Let C and D be two categories. Define the product category $C \times D$ as the category with:

- $\mathbf{Obj}(C \times D) \stackrel{\text{def}}{=} \{(X, Y) \mid X \in \mathbf{Obj}(C), Y \in \mathbf{Obj}(D)\}$ (as a class, not necessarily a set)
- $\mathbf{Hom}(C \times D) = \{(f, g) | f \in \mathbf{Hom}(C), g \in \mathbf{Hom}(D)\}$

$\mathbf{Set} \times \mathbf{Set}$: $\mathbf{Set} \times \mathbf{Set}$ is the category with:

- $\mathbf{Obj}(\mathbf{Set} \times \mathbf{Set}) \stackrel{\text{def}}{=}$ all (X, Y) where $X, Y \in \mathbf{Obj}(\mathbf{Set})$.
- $\mathbf{Hom}(\mathbf{Set} \times \mathbf{Set}) \stackrel{\text{def}}{=}$ all (f, g) where $f, g \in \mathbf{Hom}(\mathbf{Set})$.
- $\circ$: $(f, g) \circ (h, k) \stackrel{\text{def}}{=} (f \circ h, g \circ k)$
- id_X: $\mathrm{id}_{(X,Y)} \stackrel{\text{def}}{=} (\mathrm{id}_X, \mathrm{id}_Y)$

5 Bisimulation and Behavioral Equivalence

One natural question to answer is given two systems, are they behaviorally equivalent. This question can be formalized by looking at terminal objects in the category of coalgebras.

Terminal Coalgebra : Recall DEF terminal-obj.

Fix a functor F. The terminal/final coalgebra is the terminal object in the category of F-coalgebras.

Terminal Coalgebra : As it turns out, most categories have a notion of a terminal object. Here we give some examples:

$FX = A \times X$: The final coalgebra consists of $* : A^\omega \to A \times A^\omega$, which maps $(a_0, a_1, \ldots) \mapsto (a_0, (a_1, \ldots))$.
Any coalgebra $c : X \to A \times X$ has a unique coalgebra morphism $! : c \to *$, namely the map which sends x to the sequence $(a_0, a_1, \ldots) : A^\omega$ that happens after applying c infinitely often:
$c(x) = (a_0, x_1), c(x_1) = (a_1, x_2), \ldots$.

$FX = 2 \times X^A$: The final coalgebra consists of 2^{A^*}, the set of languages over A.

give bisimulation definition, coinduction, examples

5.1 Coupling

Set Element Coupling : `set-element-coupling`

Let

- $F : \mathbf{Set} \to \mathbf{Set}$,
- $n \in \omega$,
- $X \in \mathbf{Obj}(\mathbf{Set})$,
- $t_i \in FX$ for all $i \in n$.

Then $t \in F(X^n)$ is a *set element coupling* iff for all i we have $F\pi_i t = t_i$. Or equivalently if TFDC:

$$\begin{tikzcd}
* \arrow[r, "t"] \arrow[dr, "t_i"'] & FX^n \arrow[d, "F\pi_i"] \\
 & FX
\end{tikzcd}$$

5.1.1 New definitions of couplings (new)

We can generalize this definition so that it does not need to operate on set, also so that it acts on objects instead of elements, and not limit it to one object X but instead a collection of different objects:

Definition: Coupling `coupling`

A *coupling* for $\prod_{i\in I} F(X_i)$ is a morphism $c : \prod_{i\in I} F(X_i) \to F(\prod_{i\in I} X_i)$ such that TFDC:

$$\begin{tikzcd}
 & \cdots & \\
 & F_{X_{i_1}} & \\
 & F_{X_{i_0}} & \\
\prod_{i\in I} F(X_i) \arrow[uuur, bend left] \arrow[uur, "\pi_{i_1}"] \arrow[ur, "\pi_{i_0}"] \arrow[rr, "c"] & & F\left(\prod_{i\in I} X_i\right) \arrow[uuul, bend right] \arrow[uul, "F(\pi_{i_1})"'] \arrow[ul, "F(\pi_{i_0})"']
\end{tikzcd}$$

i.e. $\forall_i^I \pi_i = F\pi_i \circ c$

this relates to the earlier definition above:

Coupling $\implies$ Coupling (original) for every object : Let c be a coupling DEF `coupling` on **Set** between $X_i = X$ for all $i \in I$ for a finite I (so $|I| \in \omega$). Then for any $(t_i)_{i\in I} \in \prod_{i\in I} F(X)$ is a set element coupling DEF `set-element-coupling`.

Proof. Take $p \stackrel{\text{def}}{=} (t_i)_{i\in I} \in \prod_{i\in I} F(X)$. From the definition of coupling, we have that $\pi_i(p) = (F(\pi_i) \circ c)p = (F(\pi_i)(c(p))$. Letting $t \stackrel{\text{def}}{=} c(p)$, we get $F(\pi_i)t = F\pi_i t$. Letting $n = |I| \in \omega$ we get exactly the definition of a set element coupling. $\square$

(note this is a simpler version of a distributive law for a monad, however we do not assume F is a monad, but also specifically do this for a product functor)

Suppose now that $c_R : R \to FR$ is a bisimulation between $c_X : X \to FX$, and $c_Y : Y \to FY$. Then note because of DEF coalgebra-hom, there must be some span on sets $X \xleftarrow{\phi_X} R \xrightarrow{\phi_Y} Y$, and because **Set** has products, by DEF product these must factor out into product maps. The question is, is there any way to given a bisimulation $c_R : R \to FR$, reduce it to a coalgebra of type $X \times Y \to F(X \times Y)$? this would allow for an easier way to interpret what the bisimulation is doing.

Result: Products of Coalgberas vs Products on Original Category

Let C be a category with products.
If a coupling $\gamma : FX \times FY \to F(X \times Y)$ exists, then for any two coalgebras $c_x : X \to FX, c_Y : Y \to FY$ we have:

- The product of c_X, c_Y exists in the category of F-coalgebras.
- This product can be of type $c_{X \times Y} : X \times Y \to F(X \times Y)$

Proof. Let $c_X : X \to FX$, and $c_Y : Y \to FY$, and let $c_R : R \to FR$ be a bisimulation, with homomorphisms $\phi_X : R \to X$, $\phi_Y : R \to Y$. The goal is to show that a product exists, i.e. $\exists c_{X \times Y} : X \times Y \to F(X \times Y)$ such that for any bisimulation c_R, TFDC (in the category of F-coalgebras):

$$\begin{tikzcd}
 & c_{X \times Y} \arrow[dl, "\pi_X"'] \arrow[dr, "\pi_Y"] & \\
c_X & c_R \arrow[l, "\phi_X"] \arrow[r, "\phi_Y"'] \arrow[u, dashed, "\exists!"] & c_Y
\end{tikzcd}$$

Now going back to C, notice that since C has products, there exists an $f : R \to X \times Y$ such that TFDC (in C):

$$\begin{tikzcd}
 & X \times Y \arrow[dl, "\pi_X"'] \arrow[dr, "\pi_Y"] & \\
X & R \arrow[l, "\phi_X"] \arrow[r, "\phi_Y"'] \arrow[u, dashed, "\exists! f"] & Y
\end{tikzcd}$$

□

Now note that we assume c_R is a bisimulaiton, so we have TFDC (in C)

$$\begin{tikzcd}
X \arrow[d, "c_Y"'] & R \arrow[l, "\phi_X"'] \arrow[r, "\phi_Y"] \arrow[d, "c_R"'] & Y \arrow[d, "c_Y"] \\
FX & FR \arrow[l, "F\phi_X"] \arrow[r, "F\phi_Y"'] & FY
\end{tikzcd}$$

Notice, that if we then consider the object $FX \times FY$, this is a product object in C. So it must have projection maps π_{FX}, π_{FY}. Furthermore, since $X \times Y \xrightarrow{\pi_X} X \xrightarrow{c_X} FX$ and $X \times Y \xrightarrow{\pi_Y} Y \xrightarrow{c_Y} FY$, this forms a span of FX and FY, so there must be a unique map $\exists! : X \times Y \to FX \times FY$ such that TFDC:

$$\begin{tikzcd}
X \arrow[dd, "c_X"'] & & & \\
 & R \arrow[ul, "\phi_X"'] \arrow[rr, dashed, "\exists! f"] \arrow[dr, "\phi_Y"] \arrow[dd, "c_R"'] & & X \times Y \arrow[ulll, "\pi_X"'] \arrow[dl, "\pi_Y"] \arrow[dd, dashed, "\exists! = \langle c_x, c_y \rangle"] \\
FX & & Y \arrow[dd, "c_Y"] & \\
 & FR \arrow[ul, "F\phi_X"] \arrow[dr, "F\phi_Y"'] & & FX \times FY \arrow[ulll, "\pi_{FX}"'] \arrow[dl, "\pi_{FY}"] \\
 & & FY &
\end{tikzcd}$$

Now because F is a functor, the following diagram must commute as well (it is just applying the functor everywhere on the product diagram for $X \times Y$, and functors preserve commutative diagrams)

$$\begin{tikzcd}
& F(X \times Y) \arrow[dl, "F(\pi_X)"'] \arrow[dr, "F(\pi_Y)"] & \\
FX & FR \arrow[l, "F(\phi_X)"] \arrow[r, "F(\phi_Y)"'] \arrow[u, "F(f)"] & FY
\end{tikzcd}$$

So we add it to our diagram:

$$\begin{tikzcd}
X \arrow[dd, "c_X"] & & & \\
& R \arrow[ul, "\phi_X"] \arrow[rr, dashed, "\exists! f"] \arrow[dr, "\phi_Y"] \arrow[dd, "c_R"] & & X \times Y \arrow[ulll, "\pi_X"'] \arrow[dl, "\pi_Y"] \arrow[dd, dashed, "\exists! = \langle c_x, c_y \rangle"] \\
FX & & Y \arrow[dd, "c_Y"] & \\
& FR \arrow[ul, "F\phi_X"] \arrow[dr, "F\phi_Y"] \arrow[dd, "Ff"] & & FX \times FY \arrow[ulll, "\pi_{FX}"'] \arrow[dl, "\pi_{FY}"] \\
& & FY & \\
& F(X \times Y) \arrow[uuul, "F\pi_X"] \arrow[uur, "F\pi_Y"'] & &
\end{tikzcd}$$

Now note that since we have a coupling (DEF coupling) $\gamma : FX \times FY \to F(X \times Y)$, it must commute with $\pi_X, \pi_Y, F\pi_X, F\pi_Y$ in the diagram above, so we get TFDC:

$$\begin{tikzcd}
X \arrow[dd, "c_X"] & & & \\
& R \arrow[ul, "\phi_X"] \arrow[rr, dashed, "\exists! f"] \arrow[dr, "\phi_Y"] \arrow[dd, "c_R"] & & X \times Y \arrow[ulll, "\pi_X"'] \arrow[dl, "\pi_Y"] \arrow[dd, dashed, "\exists! = \langle c_x, c_y \rangle"] \\
FX & & Y \arrow[dd, "c_Y"] & \\
& FR \arrow[ul, "F\phi_X"] \arrow[dr, "F\phi_Y"] \arrow[dd, "Ff"] & & FX \times FY \arrow[ulll, "\pi_{FX}"'] \arrow[dl, "\pi_{FY}"] \arrow[ddll, bend left, "\gamma"] \\
& & FY & \\
& F(X \times Y) \arrow[uuul, "F\pi_X"] \arrow[uur, "F\pi_Y"'] & &
\end{tikzcd}$$

Thus finally we see that there is a coalgebra $c_{X \times Y} \overset{\text{def}}{=} \gamma \circ \langle c_x, c_y \rangle : X \times Y \to F(X \times Y)$, and there are homomorphisms $f : c_R \to c_{X \times Y}$, $\pi_X : c_{X \times Y} \to c_X$, and $\pi_Y : c_{X \times Y} \to c_Y$.
Now to show that the coalgebra homomorphism f (not f on **Set**, but f as a coalgebra homomorphism) is unique, simply note that any coalgebra homomorphism $p : c_R \to c_{X \times Y}$ must be a map $f : R \to X \times Y$, and since on **Set** this is unique, it must also be unique.

6 Coalgebra of MDPs (new)

Recall the MDP functor:

Definition: MDP functor `mdp-functor`

Define the MDP functor on **Set** as:

$$F : S \to \mathbb{R} \times (\Delta S)^A \tag{12}$$

Note this will consider MDPs with state valued reward. Alternatively you could consider $F : S \to (\mathbb{R} \times \Delta S)^A$, which assigns a reward to a transition (s, a, s') instead of a state s. Much of the work we do should transfer up to some minor changes.

Result: MDP homomorphim `MDP-hom`

Consider MDP coalgebras:

$$\begin{aligned}
\mathcal{M}_X &: X \to FX && (13)\\
&: x \mapsto (r^X_x : \mathbb{R}, P^X_x : (\Delta X)^A) && (14)\\
\mathcal{M}_Y &: Y \to FY && (15)\\
&: y \mapsto (r^Y_y : \mathbb{R}, P^Y_y : (\Delta Y)^A) && (16)
\end{aligned}$$

Note that ϕ is a MDP-coalgebra homomorphism iff:

$$\begin{tikzcd}
X \arrow[r, "\phi"] \arrow[d, "\mathcal{M}_X"'] & Y \arrow[d, "\mathcal{M}_Y"] \\
FX \arrow[r, "F\phi"'] & FY
\end{tikzcd}$$

$$\begin{aligned}
\ldots \iff & (F\phi) \circ \mathcal{M}_X = \mathcal{M}_Y \circ \phi && (17)\\
\iff & \forall^X_x (F\phi) \circ \mathcal{M}_X(x) = \mathcal{M}_Y \circ \phi(x) && (18)\\
\iff & \forall^X_x (F\phi)(r^X_x, P^X_x) = (r^Y_{\phi(x)}, P^Y_{\phi(x)}) && (19)\\
\iff & \forall^X_x (r^X_x, (\Delta\phi)P^X_x) = (r^Y_{\phi(x)}, P^Y_{\phi(x)}) && (20)\\
\iff & \forall^X_x \left(r^X_x, (y : Y, a : A) \mapsto \sum_{\substack{x' \in X \\ \phi(x') = y}} P^X_x(a, x') \right) = (r^Y_{\phi(x)}, P^Y_{\phi(x)}) && (21)\\
\iff & \forall^X_x (r^X_x = r^Y_{\phi(x)}) \wedge \forall^Y_y \forall^A_a \left(\sum_{\substack{x' \in X \\ \phi(x) = y}} P^X_x(a, x') = P^Y_{\phi(x)}(a, y) \right) && (22)\\
\iff & \forall^X_x (r^X_x = r^Y_{\phi(x)}) \wedge \forall^Y_y \forall^A_a \left(P^Y(\phi(x) \xrightarrow{a} y) = P^X(x \xrightarrow{a} \phi^{-1}[\{y\}] \right) && (23)
\end{aligned}$$

6.1 Bisimulation

Recall the transition system version of bisimulation:

Definition: Standard MDP Bisimulation `standard-mdp-bisim`

For an MDP $\mathcal{M} = (S, A, P, R, \gamma)$, a bisimulation is an equivalence relation $B \subseteq S^2$ such that:

$$sBs' \iff R(s) = R(s') \wedge \forall^A_a \forall^{S/B}_C P[s \xrightarrow{a} C] = P[s' \xrightarrow{a} C] \tag{24}$$

And bisimilarity is the union of all such B.

Now conversely, when you look at the coalgebraic version, some natural questions arise:

1. How can you get a relation out of a span?
2. If you take the relation, also assume $c_y = c_x$, does this definition correspond to the standard one?

For question 1, the natural choice for a relation is $\{(\phi_x(b), \phi_y(b)|b \in R\}$. Or equivalently using categories, simply take the unique map f to the product, and the image of that map:

$$\begin{tikzcd} & X \times Y \arrow[dl] \arrow[dr] & \\ X & B \arrow[l, "\phi_X"'] \arrow[r, "\phi_Y"] \arrow[u, dashed, "\exists! f"'] & Y \end{tikzcd}$$

Definition: Induced Relation for **Set** bisimulation `induced-relation-bisimulation`

Let F be an endofunctor on **Set**. For a bisimulation c_B of type given below:

$$\begin{tikzcd} X \arrow[d, "c_X"'] & B \arrow[l, "\phi_0"'] \arrow[r, "\phi_1"] \arrow[d, "c_B"] & X \arrow[d, "c_x"] \\ FX & FB \arrow[l, "F\phi_0"] \arrow[r, "F\phi_1"'] & FX \end{tikzcd}$$

Let B_I denote the induced relation on X^2, namely $\{\phi_X(b), \phi_X(b)|b \in B\}$.

For 2, it is a bit more tricky. I think it can be done if you assume that the resulting relation is an equivalence relation.

We give the formal proof first, then give some intuition.

Result: Coalgebraic Bisimulation is MDP bisimulation

Suppose c_B is an MDP-coalgebraic bisimulation:

$$c_X \xleftarrow{\phi_{X_0}} c_B \xrightarrow{\phi_{X_1}} c_X$$

Such that the induced relation B_I is an equivalence relation. Then B_I is a traditional MDP bisimulation.

Proof.

Forward direction : WTS $\forall_{x,x'} xB_I x' \implies r^X(x) = r^X(x') \wedge \forall_a^A \forall_C^{X/B_I} P^X[x \xrightarrow{a} C] = P^X[x' \xrightarrow{a} C]$
Assume $xB_I x'$.
From DEF induced-relation-bisimulation, we have $b \in B$ such that $\phi_0(b) = x$, and $\phi_1(b) = x'$.

$$\implies r^B(b) = r^X(\phi_0(b)) = r^X(\phi_1(b)) \tag{25}$$
$$= r^X(x) = r^X(x') \tag{26}$$

Take arbitrary $a \in A$, $C \in X/B_I$. Note $\phi_0^{-1}[C] = \phi_1^{-1}[C]$, because:

$\phi_0^{-1}[C] = \phi_1^{-1}[C]$: Recall $B_I = \{\phi_0(b), \phi_1(b) | b \in B\}$.
So $\phi_0(b) \in C \iff \phi_1(b) \in C$. Thus:

$$\phi_0^{-1}[C] = \{b \in B | \phi_0(b) \in C\} \tag{27}$$
$$= \{b \in B | \phi_1(b) \in C\} \tag{28}$$
$$= \phi_1^{-1}[C] \tag{29}$$

Using the above, and RES MDP-hom:

$$\forall_c^C P^B[b \xrightarrow{a} \phi_0^{-1}[\{c\}]] = P^X[x \xrightarrow{a} c] \tag{30}$$
$$\implies P^B[b \xrightarrow{a} \phi_0^{-1}[C]] = P^X[x \xrightarrow{a} C] \tag{31}$$
$$= P^B[b \xrightarrow{a} \phi_1^{-1}[C]] = P^X[x' \xrightarrow{a} C] \tag{32}$$

Thus $P^X[x \xrightarrow{a} C] = P^X[x' \xrightarrow{a} C]$, and $r^X(x) = r^X(x')$.

Backward direction : WTS $\forall_{x,x'} x' B_I x \implies r^X(x) = r^X(x') \wedge \forall_a^A \forall_C^{X/B_I} P^X[x' \xrightarrow{a} C] = P^X[x \xrightarrow{a} C]$

Prove

$\square$

Intuition for Bisimulation : Now, notice that since B_I is an equivalence relation, you can view it as a matrix of size $|X| \times |X|$, where each entry corresponds to elements of $B_I \subseteq X^2$. Furthermore, you can reorder the indecies so that the matrix is block diagonal (using transitivity, reflexivity, symmetry of B) In the following figure, the red dots are elements of B, the green squares correspond to the actual matrix entries (so each green square corresponds to a pair $(x_0, x_1) \in B_I \subseteq X^2$, and the blue squares represent equivalence classes of B. Everything outside of a blue square will be empty.

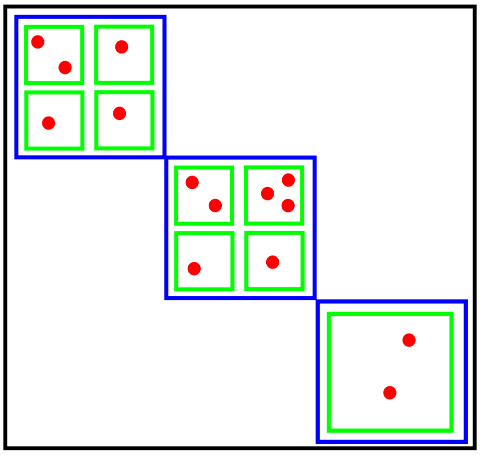


Figure 1: Bisimulation B grouped by equivalence classes

Notice that $P^B[b \xrightarrow{a} \phi_{X_0}^{-1}[\{x\}] = P^X[\phi_{X_0}(b) \xrightarrow{a} x]$ and $P^B[b \xrightarrow{a} \phi_{X_1}^{-1}[\{x'\}] = P^X[\phi_{X_1}(b) \xrightarrow{a} x']$ imply:

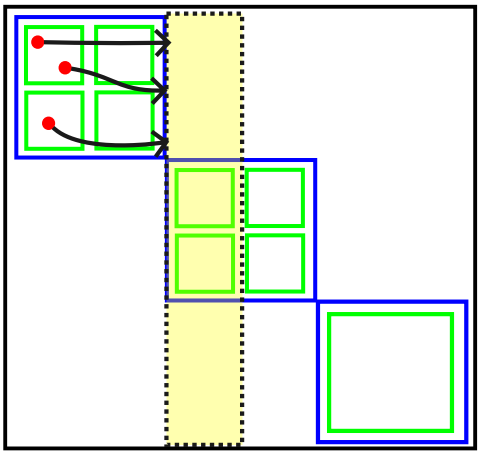


Figure 2: The Transitions from any element $b \in \phi_{X_0}^{-1}[\{x\}]$, to the entirety of $\phi_{X_0}^{-1}[\{x'\}]$

Let b be one of the red dots pictured
(i.e. some $b \in B$ s.t. $\phi_{X_0}(b) = x$)
Each transition above will happen with probability:

$$P^B[b \xrightarrow{a} \phi_{X_0}[\{x'\}]] = P^X[x \xrightarrow{a} x']$$

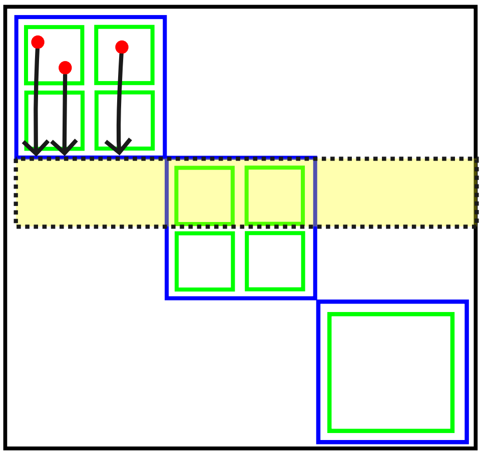


Figure 3: The transitions from any element $b \in \phi_{X_1}^{-1}[\{x\}]$, to the entirety of $\phi_{X_1}^{-1}[\{x''\}]$

Let b be one of the red dots pictured
(i.e. some $b \in B$ s.t. $\phi_{X_1}(b) = x$)
Each transition above will happen with probability:

$$P^B[b \xrightarrow{a} \phi_{X_1}[\{x'\}]] = P^X[x \xrightarrow{a} x']$$

Now let $C = \{q \mid qB_I x'\}$ be an equivalence class of B_I that includes x'.
So we can see that by summing up instances of the last picture, we get:

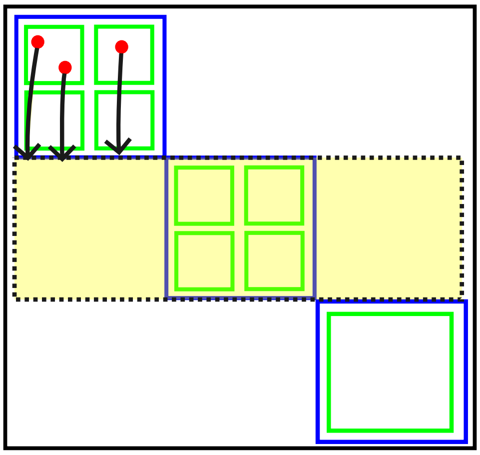


Figure 4: Transitions from any element $b \in \phi_{X_1}^{-1}[\{x\}]$, to $\phi_{X_1}^{-1}[C]$, i.e. anything related to x'

In the figure above, the probability of each of these transitions is

$$P^B\left[b \xrightarrow{a} \phi_{X_1}^{-1}[C]\right] = P^X[x \xrightarrow{a} C]$$

Likewise the same argument holds when you sum up columns instead, and you get

$$P^B\left[b \xrightarrow{a} \phi_{X_0}^{-1}[C]\right] = P^X[x \xrightarrow{a} C]$$

Note that because B_I is an equivalence relation, it must be that $\phi_{X_0}^{-1}[C] = \phi_{X_1}^{-1}[C]$.

7 LDBA/LDBW Coalgebraically (new)

7.1 Automota review

Recall:

Definition: Generalized Buchi Automoton (GBA) GBA

A Generlized Buchi Automota (GBA) is a tuple

$$\mathcal{B} = (Q, \Sigma, \delta, \alpha) \tag{33}$$

such that:

- Q is a set (states)
- Σ is a set (alphabet)
- $\delta : Q \to \Sigma \to 2^Q$ is a transition function (NDTM)
- $\alpha = \{F_1, \dots F_n\}$ with $F_i : 2^Q$ is a set of accepting conditions

Coalgebraically, we can view a GBA as $F : Q \to (2^Q)^\Sigma \times 2^n$, where the left part yields the next possible states given a letter, and the right part gives which (if any) of the acceptance conditions the current state is a part of.

Definition: Pointed GBA

A pointed GBA is a GBA $\mathcal{B} = (Q, \Sigma, \delta, \alpha)$ along with an initial state $q_0 \in Q$.

For now we will consider non pointed GBA, but threre exist pointed coalgebras, so perhaps pointed GBAs reduce to pointed coalgebras.

Definition: $\inf \tau$ for trajector τ

For a run/trajectory $\tau : (Q \times \Sigma \times Q)$, let:

$$\inf \tau \overset{\text{def}}{=} \{q | (q, \sigma, q') \in \tau \text{ occurs infinitely often in } \tau\} \tag{34}$$

Accepting Run : A run $\tau : (Q \times \Sigma \times Q)$ is accepted if $\inf \tau \circ F_i \neq 0$ for all F_i.

Definition: Limit Deterministic Buchi Automota (LDBA)

A GBA is a LDBA iff Q can be partitioned $Q = Q_I \sqcup Q_D$ such that:

- Tansitions from Q_D must remain in Q_D, and must be deterministic:
  For any $q_d : Q_D$, $\sigma : \Sigma$, $\delta(q,\sigma) \subseteq Q_D$ and $|\delta(q, \sigma)| = 1$.
- Accepting states are in Q_D:
  For any $F_i : \alpha$, $F_i \subseteq Q_D$.

Coalgebraically this is more tricky. Our goal is to find a functor F such that every F-coalgebra can be modeled by an LDBA, and likewise that every LDBA can be modeled by a F-coalgebra.

The naiive approach would be something like: $F : Q \to (2^Q)^\Sigma + (Q^\Sigma \times 2^k)$. The problem is enforcing that transitions from Q_D remain in Q_D. (otherwise you can construct a coalgebra where the output of one of the deterministic transitions is something not in Q_D). So we need to enforce that if the coalgebra "knows" that the input was in Q_D, it should only output elements in Q_D.

We can model this however by going to $\mathbf{Set}/2$, i.e. DEF slice-cat. This will consist of objects of the form $m_X : X \to 2$ for any set X, and morphisms of the form $f : m_X \to m_Y$ which make TFDC:

$$\begin{tikzcd} X \arrow[rr, "f"] \arrow[dr, "m_x"'] & & Y \arrow[dl, "m_y"] \\ & 2 & \end{tikzcd}$$

Note that any object $m_X : X \to 2$ in $\mathbf{Set}/2$ can be thought of as a partition $X = m_X^{-1}[\{0\}] + m_X^{-1}[\{1\}]$.

Furthermore any morphism $m_X \to m_Y$ must preserve the partition, i.e. $m_X^{-1}[\{0\}] = m_Y^{-1}[\{0\}]$ and $m_X^{-1}[\{1\}] = m_Y^{-1}[\{1\}]$.

Equivalently, we can think of $\mathbf{Set}/2$ as $\mathbf{Set} \times \mathbf{Set}$. Then:

Definition: LDBA Functor

8 Coalgebraic Behavioral Metrics

This section summarizes

Reference coalgebraic behavioral metrics paper

The goal of this section is to define a quantitative notion of bisimulation, namely a bisimulation (pseudo) metric for coalgebras.

The main idea: If you have some pseudometric d^X on X, and also a way to lift d^X to a pseudometric d^{FX} on FX, then

Essentially the goal is to lift a metric on X^2 to a metric on $F(X^2)$.

First some review some definitions on pseudometrics, and some notation/assumptions.

Then we cover some category theory definitions that we will use.

8.1 Metric review and notation

Definition: Pseudometric Space

A pseudometic space is a pair (X, d^X), for a set set X and function $d^X : X \times X \to [0,T]_{\mathbb{R}}$ for some $T \in \mathbb{R}$, subject to: $(\forall^X_{x,y})$

- (Symmetry) $d^X(x,y) = d^X(y,x)$
- (Triangle Inequality) $d^X(x,z) \leq d^X(x,y) + d^X(y,z)$
- (Reflexivity) $d^X(x,x) = 0$

(Note we are assuming a pseudometric is bounded)

Definition: Metric

A pseudometric space (X, d^X) is a metric space iff it satisfies Definiteness, i.e.

$$\forall^X_{x,y} d^X(x,y) \neq 0 \implies d(x,y) > 0 \tag{35}$$

i.e. equivalently (under the pseudometric axioms):

$$d^X(x,y) = 0 \iff x = y \tag{36}$$

Notation T, F : For the rest of this section, let $T \in \mathbb{R}$ be some fixed real number, and $F : \mathbf{Set} \to \mathbf{Set}$ be some endofunctor on **Set**.

8.2 Categorical Prerequisites

Definition: PMet

Define **PMet** as the category with:

- $\mathbf{Obj}(\mathbf{PMet}) \stackrel{\text{def}}{=}$ all pseudo-metric spaces (X, d^X).
- $\mathbf{Hom}(\mathbf{PMet}) \stackrel{\text{def}}{=}$ all nonexpansive functions, i.e. $f : (X, d^X) \to (Y, d^Y)$ where $d^Y \circ (f, f) \leq d^X$. (here $\leq$ between functions is the pointwise order, i.e. $f \leq g$ iff $f(x) \leq g(x)$ for all $x \in X$)
- $\circ$ is the composition of functions.
- id_X is the identity function on X.

Note that this is indeed a category, as nonexpansive functions are closed under composition, and the identity function is nonexpansive.

Definition: Lifting of endofunctor from **Set** to **PMet**

Let $F : \mathbf{Set} \to \mathbf{Set}$. A lifting of F to **PMet** is a functor $\overline{F}$ such that TFDC:

$$\begin{tikzcd}
\mathbf{PMet} \arrow[r, "\overline{F}"] \arrow[d, "U"'] & \mathbf{PMet} \arrow[d, "U"] \\
\mathbf{Set} \arrow[r, "F"'] & \mathbf{Set}
\end{tikzcd}$$

(in the category of small categories)

8.3 Evaluations

Now consider coalgebras on **Set**.

We can think of $\mathbf{Set}/[0,T]_{\mathbb{R}}$ (recall DEF `slice-cat`) as having objects that have a sort of value function $V_X : X \to [0,T]_{\mathbb{R}}$, which assings a value to each state.

Note that also any pseudometric space (X, d^X) can be thought of as an object in $\mathbf{Set}/[0,T]_{\mathbb{R}}$, namely $d^X : X \times X \to [0,T]_{\mathbb{R}}$.

Definition: Evaluation Function, Evaluation Functor

Given a fixed **Set** endofunctor F. We want to find a way to lift F to $\mathbf{Set}/[0,T]_{\mathbb{R}}$:

Evaluation Function `eval` : An evaluation function is any F algebra with domain $[0,T]_{\mathbb{R}}$, i.e. any map:

$$\texttt{eval} : F([0,T]_{\mathbb{R}}) \to [0,T]_{\mathbb{R}} \tag{37}$$

Evaluation Functor $\tilde{F}$: Given a fixed `eval`, $\tilde{F}$ is an endofunctor on $\mathbf{Set}/[0,T]_{\mathbb{R}}$ that takes:

- On objects: $V_X \mapsto \texttt{eval} \circ FV_X$
- On morphisms: $f \mapsto Ff$

This means you can lift any diagram in the slice category. Pictorially, this means that you can apply $\tilde{F}$ everywhere to a diagram:

$$\begin{tikzcd}
X \arrow[rr, "f"] \arrow[dr, bend right, "V_X"'] & & Y \arrow[dl, bend left, "V_Y"] \\
& {[0,T]_{\mathbb{R}}} &
\end{tikzcd}
\quad \mapsto \quad
\begin{tikzcd}
FX \arrow[rr, "\tilde{F}f \stackrel{\text{def}}{=} Ff"] \arrow[dr, dashed, "V_X"] \arrow[ddr, bend right, "\stackrel{\text{def}}{=} \tilde{F}V_X"'] & & FY \arrow[dl, dashed, "V_Y"'] \arrow[ddl, bend left, "\stackrel{\text{def}}{=} \tilde{F}V_Y"] \\
& F[0,T]_{\mathbb{R}} \arrow[d, dashed, "\texttt{eval}"] & \\
& {[0,T]_{\mathbb{R}}} &
\end{tikzcd}$$

8.4 Kantorovic Metric

Definition: Kantorovic Metric

Given F, `eval`, then for any pseudometric (X,d) define d_K:

$$d_{K(X,d^X)}(t_1,t_2) \overset{\text{def}}{=} \sup\left\{ d^{[0,T]_{\mathbb{R}}}(\tilde{F}f(t_1),\tilde{f}(t_2)) \middle| f : \mathbf{Hom}^{\mathbf{PMet}}((X,d),([0,T]_{\mathbb{R}},d^{[0,T]_{\mathbb{R}}})) \right\} \tag{38}$$

$$= \sup\left\{ \left|\tilde{F}V_X(t_1) - \tilde{F}V_X(t_2)\right| \middle| V_X : X \to [0,T]_{\mathbb{R}},\ V_X \text{ non expansive} \right\} \tag{39}$$

Pictorially, it is sup over all non expansive functions $V_X : (X,d^X) \to ([0,T]_{\mathbb{R}}, d^{[0,T]_{\mathbb{R}}})$ such that TFDC:

$$\begin{tikzcd}
FX \times FX \arrow[r, rightsquigarrow, "\sup_{V_X} FV_X \times FV_X"] \arrow[d, dashed, "d_K"'] & F[0,T]_{\mathbb{R}} \times F[0,T]_{\mathbb{R}} \arrow[d, "\mathtt{eval}\times\mathtt{eval}"] \\
{[0,T]_{\mathbb{R}}} & {[0,T]_{\mathbb{R}} \times [0,T]_{\mathbb{R}}} \arrow[l, "d^{[0,T]_{\mathbb{R}}}"]
\end{tikzcd}$$

where $d^{[0,T]_{\mathbb{R}}}$ is $L1$ distance on $\mathbb{R}$.

Definition: Kantorovich Lifting/Functor

The Kantorovich lifting (functor) of $F : \mathbf{Set} \to \mathbf{Set}$ is $F_K : \mathbf{PMet} \to \mathbf{PMet}$ such that $F_K(X,d^X) = (X, d_{K(X,d^X)})$, and $F_K f = Ff$.

Show examples, in particular on Δ

Useful results :

1. If (X,d) is a pseudometric, then so is d_K.
2. F_K for a functor F preserves isometries
3.

9 More complicated results

9.1 Existance of final coalgebra

(A slightly more involved result shows how to build a final coalgebra)

Definition: Limit `limit`

Fill in

Definition: Colimit `colimit`

Fill in

Cochain, Chain : A cochain is a diagram of the form:

Cochain, Chain : A cochain is a diagram of the form:

Constructing the final coalgebra : Let C be a category with terminal object $*$, and limits of all ordinal indexed cochains

Try building final coalgebra inductivly?